

第 12 章 systemd Unit、服务与依赖关系

从声明对象到真实功能：把加载来源、当前状态、持久配置、依赖顺序与服务进程放进同一条证据链。

对象模型操作语义验证诊断经典任务

大号阅读版

本章阅读导航

先抓住一条主线

systemd 不只是“启动服务”的命令集合。它先加载并合并 unit 定义，再根据依赖和顺序建立 job，最后形成当前状态、进程身份与持久启动关系。任何一层成立，都不能自动替代最终功能验收。

专题地图

1. 知识专题 Unit 类型、名称、模板与实例
2. 知识专题 Load、Active、Sub 与 UnitFile 四层状态
3. 操作专题 status、show、cat 与依赖查询
4. 操作专题 当前控制、持久启用与 mask
5. 知识专题 Requires/Wants 与 After/Before
6. 知识专题 socket、path、timer 与 Restart 激活
7. 操作专题 drop-in、自定义 service 与 daemon-reload
8. 诊断专题 从症状推进到下一条有区分度的证据
9. 经典任务 受限账号服务与重新激活诊断

阅读时持续回答

1. 现在判断的是加载、当前、持久还是功能状态？
2. 看到的是 unit、job 还是进程？
3. 依赖关系是否同时声明了顺序？
4. 服务被谁拉入事务、被谁再次激活？
5. 实际加载的是发行版主文件还是管理员 drop-in？
6. 修改后需要 daemon-reload、应用 reload 还是 restart？
7. 哪条证据只能证明局部，下一层验收是什么？

01识别 Unit类型、完整名称与实例

02确认来源主文件、drop-in 与优先级

03读取状态Load / Active / Sub / UnitFile

04展开关系依赖、顺序与激活器

05实施变更当前、持久与定义加载

06分层验收身份、功能与重启后状态

第 12 章 · 正文

Linux 上看到一个进程，并不等于已经理解了它的管理对象。进程可能由管理员在 Shell 中直接启动，也可能由 systemd 根据一个 `.service` 声明创建；服务可能由默认 target 在引导时拉起，也可能由 `.socket`、`.path` 或 `.timer` 在事件发生时激活；管理员看到的 unit 内容还可能是发行版主文件、运行时定义与多个 drop-in 合并后的结果。

本章最常见的误判是把 `active` 当成业务正确，把 `enabled` 当成当前运行，把 `After=` 当成依赖拉起，或者把 `daemon-reload` 当成应用配置 reload。为避免这些错误，本章始终沿着下面的证据链推进：

磁盘上的 unit 定义

→ manager 的加载与合并结果

→ 依赖和顺序形成的 job 事务

→ unit 当前状态与 service 进程身份

→ 持久启动配置

→ 对外功能是否真正成立

第 11 章已经建立进程、PID 与信号的前置模型；本章只在确认 service 进程身份时引用它。完整日志检索转交第 13 章，timer 的日历表达式与调度验收转交第 15 章。

概念

Unit 是 systemd 管理的声明对象，可以描述 service、socket、path、timer、target、mount 等对象。unit 文件不是“正在运行的进程”，而是 manager 用来决定如何加载、激活、停止和关联对象的配置；观察它时要同时查看完整名称、类型和实际加载来源。

概念

Job 是 manager 为 start、stop、reload 等请求建立的一次事务任务，完成后即消失。unit 是可持续存在的管理对象，job 是针对 unit 的一次动作，service 激活后才可能产生一个或多个进程；三者不能混为同一个状态。

概念

LoadState 回答 manager 是否获得了可用定义，例如 `loaded`、`not-found`、`error` 或 `masked`。它只证明定义层，不证明 unit 已启动；新建或修改文件后，磁盘内容与 manager 内存中的定义还可能因为尚未 daemon-reload 而不一致。

概念

ActiveState 与 SubState 分别描述跨类型通用生命周期和类型专用细节。active 可能对应 service 的 running，也可能对应 oneshot 的 exited；因此不能只看到 active 就推断存在长期进程，更不能据此推断端口或业务结果正确。

概念

UnitFileState 描述持久激活配置，而不是当前运行状态。enabled 表示已按 [Install] 建立链接，disabled 表示尚未启用，static 通常表示没有普通 enable 入口但仍可被依赖或触发，masked 则阻断普通激活。

概念

依赖关系与顺序关系 是两张独立的图。Wants=、Requires= 决定是否把另一个 unit 拉入事务；After=、Before= 只决定已进入同一事务的对象谁先谁后。只写 After 不会自动启动对方，只写 Requires 也不保证启动顺序。

概念

激活器 是能在事件发生时拉起其他 unit 的对象，例如 socket 收到连接、path 观察到文件变化、timer 到达触发时刻。它们解释了“service 停止后为什么又回来”；另一个独立机制是 service 自身的 Restart= 策略。

概念

Drop-in 是 管理员对现有 unit 做最小覆盖的配置片段，通常位于 /etc/systemd/system/UNIT.d/*.conf。它在发行版主文件之后合并，便于保留软件包升级路径；但列表型指令可能追加而非替换，覆盖 ExecStart= 时常需要先用空赋值清除旧列表。

操作语义以下六组入口先建立本章的接口地图。先确认作用对象，再选择参数；详细验证与边界在后续专题展开。

systemctl status / show / cat

SYNOPSIS

```
systemctl status UNIT [--no-pager] [--full]
systemctl show UNIT [-p PROPERTY,...]
systemctl cat UNIT
```

分别用于人工综合调查、读取精确属性和还原主文件与 drop-in 来源。

重要参数 / 形式

`--full / -l`

避免长行被省略，适合检查完整命令和路径。

`-p PROPERTY,...`

只输出指定属性，适合稳定判断。

`cat UNIT`

确认最终配置来自哪些文件，但仍需结合 `show` 判断 `manager` 状态。

`start / stop / restart / reload`

SYNOPSIS

```
systemctl start|stop|restart|reload UNIT
```

改变 `unit` 的当前状态；不会自动修改下一次引导时的持久配置。

重要参数 / 形式

`restart`

停止后重新启动，通常会中断当前进程。

`reload`

请求应用重读自身配置，前提是 `unit` 支持该动作。

`reload-or-restart`

支持 `reload` 时优先 `reload`，否则 `restart`。

`enable / disable / mask`

SYNOPSIS

```
systemctl enable|disable|mask|unmask [--now] [--runtime] UNIT
```

管理持久或运行时激活关系；`mask` 比 `disable` 更强，会阻断普通启动。

重要参数 / 形式

`--now`

在修改持久状态的同时改变当前状态。

`--runtime`

只写入 `/run`，重启后失效；适合临时控制，不等于持久配置。

`mask`

只在目标明确要求阻断普通激活时使用，先调查现有依赖。

list-dependencies

SYNOPSIS

```
systemctl list-dependencies [--reverse] [--all] UNIT
```

沿依赖图观察“它需要谁”或“谁需要它”，用于解释引导拉起和重新激活。

重要参数 / 形式

--reverse

反向查看依赖者，是调查“谁把它拉起来”的关键入口。

--all

显示默认可能省略的 inactive 依赖。

show -p Wants,Requires,After,Before

需要精确区分关系类型时读取属性。

systemctl edit / daemon-reload

SYNOPSIS

```
systemctl edit [--runtime] UNIT
systemctl daemon-reload
```

edit 创建管理员 drop-in；daemon-reload 让 manager 重新扫描并合并 unit 定义。

重要参数 / 形式

edit UNIT

默认写入持久管理员目录，优先于直接修改 vendor 文件。

edit --runtime UNIT

创建仅本次引导有效的 drop-in。

daemon-reload

只刷新 systemd 定义，不等于应用自身的 reload，也不会自动重启服务。

自定义 service / target

SYNOPSIS

```
[Service]
Type=... ExecStart=... User=... Restart=...
[Install]
WantedBy=multi-user.target

systemctl get-default
systemctl set-default TARGET
```

service 字段定义进程生命周期和身份；target 组织依赖并决定默认引导目标。

重要参数 / 形式

Type=

决定 systemd 如何判断启动完成和主进程。

ExecStart=

定义启动命令，默认不经过 Shell 解析。

User=

指定服务进程身份，必须同时检查目录访问能力。

Restart=

定义退出后的重启策略，不等同于 socket/path/timer 激活。

WantedBy=

说明 enable 时把链接放入哪个 target 的 wants 目录。

版本与环境边界 本章示例以 RHEL 9 课程与对应手册语义为基线。实际执行时应在目标主机核对 systemd 版本、unit 来源与应用行为；静态定义检查不能代替启动、跨重启和功能验收。

[知识专题] Unit 是声明对象：先分清类型、名称和实例

进入 systemd 时，最容易出现的第一类混淆是把 unit、进程和一次操作任务当成同一个东西。unit 是可持久描述的对象；systemd 为启动或停止 unit 建立 job；service 激活后才可能产生一个或多个进程。先把三者分开，才能正确理解状态、依赖和错误。

① [知识点] unit、job 与进程分别回答不同问题

- unit：systemd 管理的声明对象，例如 `sshd.service`；
- job：manager 为 start、stop、reload 等操作建立的事务任务，完成后消失；
- 进程：service 激活后产生的运行实例，由 PID 和 cgroup 表示。

一个 service 可以没有长期进程，例如已经完成的 `Type=oneshot`；一个 service 也可以拥有主进程和多个工作进程。反过来，一个普通进程如果没有归属于某个 service，也不能仅凭命令名推断应该用哪个 unit 控制。

```
systemctl status sshd.service --no-pager --full
systemctl show sshd.service -p MainPID,ControlPID,ControlGroup
```

`MainPID=0` 不一定说明 unit 不存在；对一次性任务、已退出任务或没有可识别主进程的类型，应结合 `ActiveState`、`SubState`、`Type` 与 `Result` 判断。

② [知识点] unit 后缀表示管理对象，不只是文件扩展名

后缀	管理对象	本章深度
<code>.service</code>	长期进程或一次性动作	完整展开

后缀	管理对象	本章深度
<code>.socket</code>	网络或 IPC socket	作为服务激活入口展开
<code>.path</code>	文件或目录事件	作为服务激活入口展开
<code>.timer</code>	日历或相对时间	只讲激活关系，时间表达式归第 15 章
<code>.target</code>	一组依赖与同步点	展开默认目标和依赖组织
<code>.mount</code> / <code>.automount</code>	挂载对象	只识别，主归属存储章节
<code>.device</code> / <code>.swap</code>	设备与交换空间	只识别
<code>.slice</code> / <code>.scope</code>	cgroup 层级和外部进程集合	只识别，资源治理不展开

unit 名称与对象名称之间常有规则。例如挂载点 `/var/lib/data` 对应的 mount unit 名通常需要转义；这类名称转换应使用 `systemd-escape` 辅助，而不是手工猜测。

③ [知识点] 完整 unit 名可避免对象歧义

在许多 `systemctl` 命令中省略 `.service` 时，`systemd` 会尝试补全 service 后缀：

```
systemctl status sshd
systemctl status sshd.service
```

两种形式通常指向同一对象，但在讲义、脚本和诊断记录中优先写完整名称。完整名称可以清楚区分 `foo.service`、`foo.socket` 和 `foo.timer`，也便于一次查询相关 unit：

```
systemctl status report-gateway.service report-gateway.socket
```

④ [知识点] 模板 unit 与实例共享结构，但拥有独立状态

模板名称包含 `@`：

```
worker@.service
```

实例把实例名放在 `@` 与后缀之间：

```
worker@blue.service
worker@green.service
```

模板中可使用 `%i` 获取 unit 名称中已经转义的实例标识符，使用 `%I` 获取反转义后的实例名。每个实例是独立 unit，有自己的状态、cgroup、主 PID 和失败结果。对模板进行 enable 时，还要确认 `[Install]` 是否定义了 `DefaultInstance=` 或题目是否明确要求具体实例。

⑤ [边界] 别名、生成 unit 和 transient unit 仍属于加载模型

软件包可能通过 `Alias=` 或符号链接提供别名；`/etc/fstab`、设备事件或 `generator` 可以在运行时产生 unit；`systemd-run` 可以创建 transient unit。这些对象未必对应管理员手写的固定文件，因此“在 `/usr/lib/systemd/system` 没找到同名文件”不能直接证明 unit 不存在。

最有区分度的证据是：

```
systemctl show UNIT -p LoadState,FragmentPath,SourcePath,UnitFileState
systemctl cat UNIT
```

[Cheatsheet] unit 是声明对象，job 是一次管理事务，进程是运行实例；写完整后缀避免歧义；模板 `name@.service` 与实例 `name@instance.service` 分开；找不到固定文件时继续查 `FragmentPath`、生成来源和别名。

[知识专题] 四层状态模型：Load、Active、Sub 与 UnitFile

`systemctl status` 的一行输出经常同时出现 `loaded`、`enabled` 和 `active (running)`。这些词不是同义词，而是从不同层描述同一个 unit。考试和工作排错中，最重要的习惯之一，就是先说清楚正在判断哪一层。

① [知识点] `LoadState` 回答“manager 是否获得了有效定义”

常见值包括：

- `loaded`：定义已成功加载；
- `not-found`：未找到 unit 定义；
- `error`：加载或解析发生错误；
- `masked`：名称被屏蔽，通常解析到 `/dev/null`。

`LoadState=loaded` 只说明定义可用，不说明 unit 已经启动。修改或新建 unit 文件后，如果 manager 尚未重新读取定义，磁盘内容与内存中的加载结果也可能不一致。

② [知识点] `ActiveState` 是跨类型的通用生命周期

常见通用状态：

ActiveState	含义
<code>active</code>	unit 已达到其活动定义
<code>inactive</code>	unit 当前不活动
<code>activating</code>	正在启动或等待就绪
<code>deactivating</code>	正在停止

ActiveState	含义
failed	最近一次活动过程失败并被记账
reloading	正在执行 reload

对 service 来说，`active` 仍不能自动证明端口可访问、文件持续更新或应用逻辑正确。

③ [知识点] SubState 只能在 unit 类型上下文中解释

SubState 细化类型专用状态：

- service 常见 `running`、`exited`、`dead`、`failed`；
- socket 常见 `listening`；
- timer 常见 `waiting`；
- mount 常见 `mounted`。

`active` (`exited`) 常见于成功完成并保持活动语义的 `oneshot` unit，或兼容脚本式 unit。它不能被解释为“存在一个正在运行的守护进程”。需要长期主进程时，应继续查 `Type` 和 `MainPID`。

④ [知识点] UnitFileState 回答持久激活配置，而不是当前状态

本章重点训练四个结果：

- `enabled`：按 `[Install]` 建立了持久依赖链接；
- `disabled`：可启用，但目前没有对应启用链接；
- `static`：通常没有可供普通 `enable` 使用的安装声明，仍可被依赖、触发或手工激活；
- `masked`：unit 被强制阻断激活。

环境中还可能看到 `alias`、`indirect`、`generated`、`transient` 等结果。不要把所有非 `enabled` 都解释成 `disabled`，更不能把 `static` 当作失败。

⑤ [知识点] failed 是 manager 的失败记账，不是根因本身

失败信息应继续拆分：

```
systemctl show UNIT \
    -p ActiveState,SubState,Result,ExecMainCode,ExecMainStatus
```

- `Result` 表示 systemd 对 unit 运行结果的分类；
- `ExecMainCode` 与 `ExecMainStatus` 描述主进程退出方式和状态；
- `status` 中的近期日志只能作为入口，完整日志检索转到第 13 章。

修复定义或应用问题后，`reset-failed` 可以清除 `failed` 记账和相关启动速率计数，但它不修复根因，也不自动启动服务。

⑥ [验证点] 将状态拆成五层验收矩阵

验收层	推荐证据	能证明什么	不能证明什么
定义层	<code>cat</code> 、 <code>show FragmentPath/DropInPaths</code> 、 <code>verify</code>	实际加载来源和静态定义	运行时一定成功
当前层	<code>is-active</code> 、 <code>show ActiveState/SubState/Result</code>	systemd 当前视角	重启后自动启动、业务正确
持久层	<code>is-enabled</code> 、目标链接	引导依赖配置	当前已经运行
身份层	<code>show MainPID + ps</code>	实际 UID、命令和进程归属	对外功能正确
功能层	端口、请求、文件或数据检查	用户真正需要的终态	其他持久层自动正确

[Cheatsheet] `LoadState` 看定义能否加载；`ActiveState` 看通用当前状态；`SubState` 看类型细节；`UnitFileState` 看持久配置；failed 要继续查 `Result` 和退出状态；最终验收必须分层。

[操作专题] 用 `systemctl` 建立可判定的 Unit 证据视图

稳定的 `systemd` 调查不是先执行 `restart`，而是依次回答：对象是否存在、当前处于什么状态、采用了哪份定义、谁依赖或触发它。不同子命令回答不同问题，混用会产生错误推断。

① [操作] 区分已加载对象与磁盘上的 unit file

作用对象：manager 已加载的 unit，或搜索路径中的 unit 文件。

基本语义：`list-units` 查看当前已加载对象；`list-unit-files` 查看安装文件及启用状态。

典型形式：`systemctl list-units --type=service --all`；`systemctl list-unit-files --type=service`。

验证边界：默认列表未出现不等于未安装；unit 可能 `inactive` 且未加载。

```
systemctl list-units --type=service --all
systemctl list-unit-files --type=service
systemctl list-units --state=failed
```

② [操作] `status` 用于人工调查入口

作用对象：一个或多个 unit 的综合状态。

基本语义：汇总加载来源、启用状态、当前状态、主 PID、cgroup 和少量近期日志。

典型形式：`systemctl status UNIT --no-pager -l`。

关键参数: `--no-pager` 直接输出; `-l` 避免长行截断。

验证边界: `status` 适合人读, 不适合依靠颜色或整段文本做稳定脚本判断。

```
systemctl status report-gateway.service --no-pager -l
systemctl status report-gateway.service report-gateway.socket --no-pager -l
```

③ [操作] `is-active`、`is-enabled` 和 `is-failed` 用于单一判断

```
systemctl is-active report-gateway.service
systemctl is-enabled report-gateway.service
systemctl is-failed report-gateway.service
```

这些命令通过退出状态和短文本回答单一问题, 适合验收脚本。但三者仍需分别执行: `active` 不能代替 `enabled`, `not failed` 也不能证明 `active` 或功能正确。

④ [操作] `show -p` 读取 `manager` 的精确属性

作用对象: `systemd` 合并后的 `unit` 属性。

基本语义: 以 `NAME=VALUE` 输出稳定字段, 适合定位和自动检查。

典型形式: `systemctl show UNIT -p PROPERTY,...`。

关键属性: `LoadState`、`ActiveState`、`SubState`、`UnitFileState`、`MainPID`、`Result`、`FragmentPath`、`DropInPaths`、`TriggeredBy`、`Restart`。

```
systemctl show report-gateway.service \
-p LoadState,ActiveState,SubState,UnitFileState,MainPID,Result

systemctl show report-gateway.service \
-p FragmentPath,DropInPaths,Wants,Requires,After,Before,TriggeredBy,Restart
```

⑤ [操作] `cat` 还原主文件与 `drop-in` 的可读来源

`systemctl cat UNIT` 按 `manager` 识别的来源显示主文件和 `drop-in`:

```
systemctl cat report-gateway.service
```

它适合回答“管理员覆盖是否真的存在”和“当前读取的是哪个文件”。但 `cat` 显示的是文件片段, 不一定直观表示所有合并后的属性; 对列表重置、隐式依赖和生成属性, 继续使用 `show`。

⑥ [操作] `list-dependencies` 从正向和反向查看关系

```
systemctl list-dependencies report-gateway.service
systemctl list-dependencies --reverse report-gateway.service
systemctl list-dependencies --after report-gateway.service
systemctl list-dependencies --before report-gateway.service
```

- 默认正向回答“它依赖哪些对象”；
- `--reverse` 回答“谁依赖或可能拉起它”；
- `--after/--before` 从排序角度查看关系。

依赖树可能很大，必须围绕故障 unit 收缩。不要把一张完整系统树直接当作根因。

[Cheatsheet] `list-units` 看已加载对象；`list-unit-files` 看磁盘与启用状态；`status` 做人工入口；`is-*` 做单一判断；`show -p` 读精确属性；`cat` 看配置来源；`list-dependencies --reverse` 查谁会拉起它。

[操作专题] 当前控制、持久启用与强制阻断必须分开

服务管理命令可以改变当前活动状态，也可以改变下次引导时的依赖配置。两个维度互不替代。考试中的“现在启动并设置开机自动启动”实际上包含两个目标；“禁止服务再被依赖拉起”又比普通 `disable` 更强。

① [操作] `start`、`stop`、`restart` 与 `reload` 改变当前状态

```
systemctl start httpd.service
systemctl stop httpd.service
systemctl restart httpd.service
systemctl reload httpd.service
systemctl reload-or-restart httpd.service
```

- `start` 激活 unit；
- `stop` 请求停止 unit；
- `restart` 停止后重新启动，通常会替换进程并造成中断窗口；
- `reload` 只在 unit 或应用定义了重载能力时可用；
- `reload-or-restart` 在不能 `reload` 时回退到 `restart`。

修改应用配置前应优先使用应用自己的语法检查，例如 `httpd -t`。无调查地 `restart` 不能代替配置核对。

② [边界] `stop service` 不一定消除所有激活入口

停止 `service` 只改变该 unit 当前状态。如果 `socket`、`path`、`timer`、依赖 unit 或外部管理器仍然活动，服务可能稍后再次被激活。遇到“stop 后又起来”，先查：

```
systemctl show UNIT -p TriggeredBy,Restart,PartOf
systemctl list-dependencies --reverse UNIT
```

这与第 11 章的“进程被重新创建”不同：本章从 unit 的触发和重启策略解释管理者行为，不通过反复 kill 进程解决。

③ [操作] enable 与 disable 只改变持久安装链接

```
systemctl enable chronyd.service
systemctl disable chronyd.service
```

`enable` 根据 unit 的 `[Install]` 声明建立链接，例如把 unit 加入 `multi-user.target.wants/`；`disable` 移除由 `enable` 创建的链接。默认情况下，两者不启动或停止当前服务。

```
systemctl enable --now chronyd.service
systemctl disable --now chronyd.service
```

`--now` 把持久操作与当前 start/stop 组合。操作后仍分别验证：

```
systemctl is-enabled chronyd.service
systemctl is-active chronyd.service
```

④ [知识点] static 不能通过普通 enable 解决

`static` 通常表示 unit 没有 `[Install]` 安装声明，或不提供普通 `enable` 入口。它仍可能：

- 被某个 target 的依赖直接拉起；
- 由 socket/path/timer 激活；
- 被其他 service 的 Wants/Requires 拉起；
- 被管理员手工 start。

题目要求“开机自动运行 static unit”时，应先调查它设计上由谁拉起，而不是机械添加一个不理解的 `[Install]`。

⑤ [操作] mask 阻止普通激活，强度高于 disable

```
systemctl stop legacy-mail.service
systemctl mask legacy-mail.service
systemctl is-enabled legacy-mail.service
```

`mask` 通常使 unit 名称解析到 `/dev/null`，从而阻止手工、依赖和触发激活。边界：

- `mask` 不应被当作普通“不开机启动”；

- mask 不一定自动停止已经运行的 unit，应明确 stop 或使用支持的 `--now`；
- unmask 只恢复可激活能力，不会自动 enable 或 start；
- 发现 vendor unit 被 mask 时，先调查原因再解除。

⑥ [操作] reset-failed 清理记账，不修复故障

服务连续快速失败可能触发启动速率限制。完成最小修复后可执行：

```
systemctl reset-failed report-agent.service
systemctl start report-agent.service
```

`reset-failed` 清除 failed 状态和相关计数，但不会修正错误路径、权限、语法或应用配置。必须先建立证据并修复根因。

⑦ [验证点] 操作完成后按目标逐层验证

```
systemctl is-active UNIT
systemctl is-enabled UNIT
systemctl show UNIT -p MainPID,User,Result
```

然后根据服务功能验证端口、请求、文件或数据。`systemctl start` 返回 0 只说明 manager 接受并完成了该操作，不代表应用已经通过所有功能验收。

[Cheatsheet] 当前状态用 start/stop/restart/reload；持久配置用 enable/disable，双目标加 `--now`；static 查激活者；强制禁止才 mask；修复后可 reset-failed；任何命令成功都要回到当前、持久、身份和功能验证。

[知识专题] 依赖强度与启动顺序是两张独立的图

systemd 可以并行启动互不冲突的 unit。为了决定“哪些对象进入同一事务”和“它们按什么先后执行”，systemd 分别维护需求关系和顺序关系。把 `Requires=` 直接解释为“在它之后启动”，是本章必须纠正的高频误区。

① [知识点] `wants=` 表示较弱的需求关系

当 A 写有：

```
[Unit]
Wants=B.service
```

启动 A 时，B 通常也会被加入启动事务；但 B 启动失败不会自动让 A 必然失败。它适合“希望一起提供，但不是 A 建立基本功能的硬前提”的对象。

② [知识点] Requires= 表示更强的生命周期需求

```
[Unit]
Requires=B.service
```

启动 A 时，B 会被加入事务。B 被显式停止或失活时，A 通常也会受到需求关系影响。若要让 B 启动失败阻止 A 继续进入活动状态，通常还需要与 `After=B.service` 配合，确保 A 等待 B 的启动结果。

因此，“Requires 比 Wants 强”是正确起点，但不能简化成“无论顺序如何，B 失败时 A 永远不可能启动”。

③ [知识点] After= 与 Before= 只定义顺序

```
[Unit]
After=network-online.target
```

这只表示两个 unit 都在同一事务中时，本 unit 的启动动作排在对方之后；它不会单独把 `network-online.target` 拉入事务。关闭时顺序反向：启动时 After 的 unit 通常先停止。

`Before=` 是同一顺序边的反向表达。不要同时在两个 unit 中重复写互相矛盾的顺序。

④ [知识点] 常见组合同时表达“拉入”和“等待”

```
[Unit]
Wants=network-online.target
After=network-online.target
```

这表示希望网络在线目标进入事务，并在其之后启动当前服务。是否应该使用 `network-online.target` 取决于程序是否真的要求“网络已经配置到可用状态”；普通本地服务不应无差别依赖它。

⑤ [知识点] [Unit] 与 [Install] 解决不同阶段的问题

- `[Unit]` 中的 `Wants/Requires/After/Before` 是运行时依赖和顺序；
- `[Install]` 中的 `WantedBy=`、`RequiredBy=`、`Alias=` 用于 enable 时建立链接；
- `[Install]` 本身不在每次启动时直接参与运行时解析，它是安装动作的说明。

例如：

```
[Install]
WantedBy=multi-user.target
```

执行 enable 后，通常形成由 `multi-user.target.wants/` 指向该 service 的链接，于是引导进入目标时能够拉起服务。

⑥ [知识点] 隐式和默认依赖也会进入最终图

socket 与同名 service、timer 与被触发 service、mount 与父路径、target 的默认依赖等都可能自动建立关系。最终判断应以 manager 合并结果为准：

```
systemctl show UNIT -p Wants,Requires,After,Before
systemctl list-dependencies UNIT
systemctl list-dependencies --reverse UNIT
```

⑦ [诊断点] “服务太早启动”要同时检查两张图

症状：服务在网络、挂载或准备任务完成前启动并失败。

当前证据：status/show 中服务启动失败

- 假设一：前置对象根本没有进入事务
- 证据：Wants/Requires 与反向依赖
- 假设二：对象进入事务但没有排序
- 证据：After/Before
- 最小修复：补充真正需要的需求和顺序，不无差别依赖大 target
- 再验证：重新加载、重启相关 unit、核对事务和功能

[Cheatsheet] Wants/Requires 决定“是否一起做”；After/Before 决定“谁先谁后”；After 不会拉起对象；Requires 的失败传播要结合排序理解；[Install] 只指导 enable；最终图用 show 与依赖查询确认。

[知识专题] 服务为什么会回来：Socket、Path、Timer 与 Restart

看到 service 从 inactive 再次变成 active 时，不能只说“systemd 自动重启了”。重新激活至少有四种不同来源：连接触发、路径事件、时间触发、进程退出后的重启策略，以及其他 unit 的依赖操作。区分来源，才能选择最小修复。

① [知识点] socket 激活把“监听入口”和“工作进程”分开

.socket unit 可以先占有网络或 IPC socket，在连接到来时启动对应 service。优点是服务可以按需启动，并由 systemd 统一管理监听入口。

```
systemctl status report-gateway.socket report-gateway.service
systemctl show report-gateway.service -p TriggeredBy
```

只停止 service 而保持 socket listening，下一次连接就可能再次激活 service。

② [知识点] path 激活根据文件系统事件启动 service

.path unit 可以监视路径存在、变化或目录内容变化。典型场景是“某个队列目录出现新文件后执行处理服务”。服务停止后，只要 path unit 仍 active，下一次事件仍会拉起它。

③ [知识点] timer 激活 service，但时间表达式属于第 15 章

`.timer` unit 通常激活同名或明确指定的 `.service`。本章只训练：

- timer 与 service 是两个独立 unit；
- 停止 service 不会停止 timer；
- 查再激活原因时应检查 timer；
- 完整 `OnCalendar=`、`Persistent=`、随机延迟和验收归“一次性任务、周期任务与 systemd Timer”。

④ [知识点] `Restart=` 处理进程退出，不是事件激活

常用策略：

- `no`：不自动重启；
- `on-failure`：非零退出、信号或超时等失败条件下重启；
- `always`：无论正常或异常退出都重启，但正常的 manager stop 不应被理解为进程自行退出；
- `on-abnormal`、`on-abort` 等用于更细边界。

`RestartSec=` 控制尝试间隔。重启策略还受启动速率限制影响。人工执行 `systemctl stop` 的语义与进程崩溃不同，不能用 `kill MainPID` 模拟正常停服。

⑤ [操作] 用属性把激活来源分开

```
systemctl show report-gateway.service \
    -p ActiveState,SubState,TriggeredBy,Restart,RestartUsec,NRestarts

systemctl list-dependencies --reverse report-gateway.service
systemctl status report-gateway.socket report-gateway.path report-gateway.timer
```

`NRestarts` 可帮助判断 manager 是否根据 `Restart` 重启，但属性可用性和展示形式需要在目标 RHEL 9 环境确认；本章不编造具体输出。

⑥ [边界] 最小修复只关闭真正不需要的入口

若要求“停止按连接启动，但仍允许管理员手工启动 service”，应 `stop/disable socket`，而不是 `mask service`。若要求“无论依赖还是手工都不得启动”，才考虑 `mask`，并先评估依赖影响。

只停 service	→ 当前进程停止，触发入口可能仍在
停并禁用 socket/path	→ 关闭指定事件入口，service 仍可手工启动
修改 Restart 策略	→ 改变进程退出后的 manager 行为
mask service	→ 强制阻断普通激活，影响最大

[Cheatsheet] stop 后再 active：查 `TriggeredBy`、反向依赖和 `Restart`；`socket/path/timer` 是外部事件入口，`Restart` 是进程退出策略；关闭哪个入口取决于目标，不默认 `mask` 全部。

[知识专题] Unit 从哪里加载：搜索路径、优先级和合并结果

真实服务器中的 unit 经常不是一个文件。软件包提供主文件，管理员添加 drop-in，生成器根据其他配置产生运行时 unit，某些工具再创建 transient 定义。排错的核心不是背出所有目录，而是理解“高优先级来源覆盖低优先级来源，drop-in 在主文件之后合并”。

① [知识点] 三个常用系统目录构成管理员心智模型

/etc/systemd/system	管理员持久配置，优先级高
/run/systemd/system	当前启动周期的运行时配置
/usr/lib/systemd/system	软件包或发行版提供的 vendor unit

RHEL 系统的完整搜索路径还可能包含 generator、transient 和 control 目录。需要确认当前环境时使用：

```
systemd-analyze unit-paths
```

② [知识点] 同名完整 unit 按高优先级来源替换

如果 /etc/systemd/system/foo.service 存在，它会遮蔽较低优先级的同名 vendor 主文件。完整复制 vendor unit 到 /etc 虽然有效，但会让后续软件包新增指令不再自动进入管理员副本，因此只在 drop-in 无法表达完整结构替换时采用。

③ [知识点] drop-in 在主文件之后按名称合并

常见位置：

```
/etc/systemd/system/foo.service.d/*.conf
/run/systemd/system/foo.service.d/*.conf
/usr/lib/systemd/system/foo.service.d/*.conf
```

高优先级目录中的同名 drop-in 覆盖低优先级同名文件；多个不同名称的 drop-in 按词法顺序合并。管理员应使用清晰前缀，例如 10-restart.conf、20-environment.conf，但不要依赖难以审计的文件名技巧隐藏语义。

④ [知识点] FragmentPath 和 DropInPaths 是实际来源证据

```
systemctl show foo.service -p FragmentPath,DropInPaths,SourcePath
systemctl cat foo.service
```

- **FragmentPath** 指向主 unit 来源；
- **DropInPaths** 列出 manager 采用的 drop-in；
- **SourcePath** 在由其他配置生成时可能提供来源线索。

⑤ [知识点] unit 定义与应用配置是两套加载链

```
/etc/systemd/system/foo.service.d/override.conf → systemd manager 读取
/etc/foo/foo.conf                               → foo 进程读取
```

修改前者需要 manager 重新读取 unit 定义；修改后者通常需要应用 reload/restart。不要因为应用配置变化就无条件 `daemon-reload`，也不要 unit 变化后只重启应用却忽略 manager 的旧定义。

⑥ [诊断点] 磁盘内容与 manager 状态不一致时先确认 reload

症状：已经修改 `ExecStart=` 或 `User=`，但服务仍按旧方式运行。

```
当前证据: cat 文件看到新内容, show 属性或进程仍是旧内容
→ 假设: manager 尚未重新加载, 或修改了低优先级被覆盖的文件
→ 下一证据: systemctl cat; show FragmentPath/DropInPaths; unit-paths
→ 最小修复: 修改正确来源并 daemon-reload
→ 再验证: cat/show 后 restart 服务, 核对 PID、UID 和功能
```

[Cheatsheet] 管理员持久配置在 `/etc`，运行时在 `/run`，vendor 在 `/usr/lib`；完整 unit 替换风险高，局部变更优先 drop-in；实际来源看 `FragmentPath/DropInPaths`；unit 与应用配置有不同 reload 链。

[操作专题] 用 Drop-in 做最小覆盖，并让 Manager 读取新定义

直接编辑 vendor unit 会把本地修改与软件包内容混在一起，升级时难以审计。更稳定的方式是先读取最终定义，再使用 drop-in 只覆盖必要字段，完成静态检查、重新加载和分层验证。

① [操作] 变更前保存定义基线

```
systemctl cat report-agent.service
systemctl show report-agent.service \
    -p FragmentPath,DropInPaths,Type,User,ExecStart,Restart
```

基线回答：当前使用哪个主文件、已有多少 drop-in、准备覆盖的属性是什么。不要在不知道原始 `ExecStart=` 的情况下直接写覆盖。

② [操作] systemctl edit 创建管理员 drop-in

```
systemctl edit report-agent.service
```

编辑器中只写需要覆盖的部分：

```
[Service]
Restart=on-failure
RestartSec=5s
```

默认目标路径通常是：

```
/etc/systemd/system/report-agent.service.d/override.conf
```

`systemctl edit --runtime` 会写入运行时目录，重启后消失；普通考试持久变更不应误用。`--full` 创建完整 unit 副本，只有 drop-in 无法表达需求时使用。

③ [知识点] 标量覆盖与列表型指令重置方式不同

`Restart=`、`User=` 等单值属性通常由后出现的值覆盖。`ExecStart=`、`Environment=` 等可多次出现或具有列表语义的指令，不能总靠再写一行替换。

覆盖已有 `ExecStart=` 的常见形式：

```
[Service]
ExecStart=
ExecStart=/usr/local/libexec/report-agent --config /etc/report-agent.conf
```

第一行空赋值清除旧列表，第二行建立新值。省略清空可能导致多个 `ExecStart=` 冲突，尤其对非 oneshot service。

④ [知识点] `ExecStart=` 默认不经过交互 Shell

下面写法不会自动获得 Bash 的管道、重定向、`&&`、`~` 和普通环境变量展开语义：

```
ExecStart=/usr/bin/echo hello > /var/lib/app/output
```

更稳妥的选择：

- 直接调用能够完成目标的程序及参数；
- 使用 `StandardOutput=` 等 systemd 指令；
- 复杂逻辑放入可审计脚本，再以绝对路径执行；
- 只有明确需要 Shell 语义时才写 `/bin/bash -c '...'`，并处理引用与退出状态。

⑤ [操作] 用 `systemd-analyze verify` 做静态检查

```
systemd-analyze verify /etc/systemd/system/report-agent.service
```

对 drop-in 场景，也可以把相关 unit 交给 verify，随后仍以 manager 实际加载结果为准。静态检查可以发现未知指令、缺少可执行文件、依赖引用等问题，但不能证明用户权限、运行目录、端口和业务逻辑都正确。

⑥ [操作] unit 定义变化后执行 `daemon-reload`

```
systemctl daemon-reload
systemctl cat report-agent.service
systemctl show report-agent.service -p DropInPaths,Restart,RestartUsec
```

`daemon-reload` 让 systemd 重新读取 unit 和 generator 输出。它不会自动重启已运行服务，也不会让应用读取自己的配置文件。下一步是否 restart 取决于变更字段和业务允许的中断窗口。

⑦ [操作] 撤销管理员覆盖前先确认影响

```
systemctl revert report-agent.service
```

`revert` 可删除由 systemctl 管理的本地覆盖并恢复 vendor 状态。执行前应备份需要保留的自定义内容，并确认不会同时删除其他管理员仍依赖的 drop-in。完成后重新加载、重新启动并分层验证。

[Cheatsheet] 先 cat/show 建基线；局部变更用 edit；列表型 `ExecStart=` 先空赋值；`ExecStart` 默认没有 Shell；verify 只做静态检查；unit 变更后 daemon-reload；最后 cat/show + restart + 功能验证。

[操作专题] 创建自定义 Service: Type、ExecStart、User 与 Restart

自定义 service 的目标不是“写一个能被 systemctl 接受的文件”，而是让 manager 能清楚识别主进程、运行身份、工作目录、成功与失败边界、重启策略和持久激活方式。越少依赖隐式 Shell 环境，unit 越容易验证和自动化。

① [知识点] 三个 section 各自承担一层职责

```
[Unit]
Description=...
Wants=...
After=...

[Service]
Type=...
User=...
ExecStart=...
Restart=...

[Install]
WantedBy=multi-user.target
```

- [Unit]：描述对象和运行时关系；
- [Service]：定义进程生命周期与执行环境；
- [Install]：说明 enable 时建立哪些链接。

Description= 便于人工识别，但不决定运行行为。

② [知识点] 根据程序行为选择 Type=

Type	manager 如何判断启动完成	典型对象
simple	启动 ExecStart 后立即认为已启动	前台长期程序，RHCSA 常用
exec	成功执行目标程序后才认为启动完成	支持该版本时可更早发现 exec 失败
oneshot	等待命令退出，成功后完成	初始化、检查、一次性变更
forking	期望程序 fork 并让父进程退出	传统自守护程序，常需 PIDFile
notify	程序主动发送就绪通知	原生支持 sd_notify 的服务

不要仅凭“守护进程”三个字使用 forking。现代程序若可以前台运行，通常更适合由 systemd 直接拥有主进程。

③ [操作] 为长期前台程序建立基本 service

```
# /etc/systemd/system/heartbeat-worker.service
[Unit]
Description=Heartbeat file worker
After=local-fs.target

[Service]
Type=simple
User=svcwatch
Group=svcwatch
WorkingDirectory=/var/lib/heartbeat-worker
ExecStart=/usr/local/libexec/heartbeat-worker
Restart=on-failure
RestartSec=5s

[Install]
WantedBy=multi-user.target
```

关键点：

- ExecStart 使用绝对路径；
- 目标用户必须存在；
- 工作目录和写入目录必须允许该用户访问；
- Restart=on-failure 只在失败条件下重启；

- `WantedBy` 让 `enable` 有明确目标。

④ [操作] 为一次性任务使用 `Type=oneshot`

```
[Service]
Type=oneshot
User=backup
ExecStart=/usr/local/sbin/prepare-backup
```

任务成功退出后通常回到 `inactive`。只有后续 `unit` 需要把“已经完成”作为活动状态时，才考虑 `RemainAfterExit=yes`。不要为了让 `status` 显示 `active` 而无条件添加它；真正验收应检查 `Result`、退出状态和任务产物。

⑤ [知识点] `User=` 改变执行身份，但不会自动准备资源

系统 `service` 中的 `User=svcwatch` 表示由系统 PID 1 以该账号启动进程。它不会自动：

- 创建账号；
- 创建工作目录；
- 修改文件所有权；
- 绕过传统权限或 SELinux；
- 提供交互 Shell 环境。

```
id svcwatch
namei -om /var/lib/heartbeat-worker
```

SELinux 上下文和 AVC 证据的完整处理归第 28、29 章；本章只在权限正确仍失败时指出安全层边界。

⑥ [知识点] `Restart=` 要与退出语义和启动速率一起设计

```
Restart=on-failure
RestartSec=5s
```

- 对预期长期运行的 `worker`，异常退出可自动恢复；
- 对一次性任务，盲目重启可能形成重复副作用；
- 对配置永久错误，快速重启只会制造启动风暴；
- `systemd` 的启动速率限制会在短时间多次失败后阻止继续启动。

诊断时结合 `Result`、退出状态、`NRestarts` 和失败记账，不把“服务正在反复重启”误解为高可用。

⑦ [操作] 自定义 unit 的推荐部署链

```
systemd-analyze verify /etc/systemd/system/heartbeat-worker.service
systemctl daemon-reload
systemctl enable --now heartbeat-worker.service
systemctl is-active heartbeat-worker.service
systemctl is-enabled heartbeat-worker.service
```

随后确认：

```
systemctl show heartbeat-worker.service \
    -p FragmentPath,ActiveState,SubState,UnitFileState,MainPID,User,Result

pid=$(systemctl show heartbeat-worker.service -p MainPID --value)
ps -p "$pid" -o pid,user,group,etimes,cmd
```

最后通过心跳文件时间、端口或真实请求验证功能。

⑧ [验证点] 失败时从最有区分度的层继续

症状	下一条证据
Unit not found	名称、路径、 daemon-reload、 FragmentPath
status=203/EXEC 等执行失败	ExecStart 绝对路径、权限、解释器和文件格式
status=217/USER 等身份失败	用户/组是否存在，User/Group 拼写
permission denied	目录遍历、文件权限、SELinux 证据
active 但无产物	实际 MainPID、用户、WorkingDirectory、应用行为
反复重启	Result、退出状态、Restart、启动速率

具体数字状态以目标环境的 status 和 man page 为准，讲义不伪造真实输出。

[Cheatsheet] [Unit] 管关系、[Service] 管进程、[Install] 管 enable；长期前台程序常用 simple，短任务用 oneshot；ExecStart 用绝对路径；User 不会自动准备目录；Restart 要防启动风暴；部署链 verify → daemon-reload → enable --now → 五层验收。

[知识专题] Target 与 Default Target: 组织依赖，而不是运行一个守护进程

.target unit 通常没有自己的长期进程。它提供一个可命名的依赖集合和同步点，让系统在引导、救援、图形界面或多用户环境中激活一组对象。target 的核心仍是依赖关系，不应把它理解成传统 SysV “运行级别进程”。

① [知识点] target 把一组 unit 组织成可激活目标

常见对象：

- `multi-user.target`：非图形多用户系统的主要服务集合；
- `graphical.target`：在多用户基础上加入图形环境；
- `rescue.target`、`emergency.target`：恢复环境，完整操作归第 30 章；
- `network-online.target`：表示网络已达到在线语义，由相关等待服务实现。

② [操作] 查询和设置持久默认目标

```
systemctl get-default  
systemctl set-default multi-user.target
```

`set-default` 通常改变 `default.target` 的持久链接。它不等于立即切换当前系统状态。操作后应再次 `get-default` 并检查链接来源。

③ [知识点] start target 与 isolate target 的影响不同

```
systemctl start multi-user.target
```

`start` 将目标及其依赖加入当前事务，但不主动停止所有与目标无关的 `unit`。

```
systemctl isolate multi-user.target
```

`isolate` 会启动目标并停止不属于新目标依赖图的其他 `unit`，可能断开图形会话、网络或当前管理入口。它是高影响操作，必须确认远程访问和回退方式，不能作为普通“查看目标”的命令。

④ [知识点] target 的 wants/requires 目录是 enable 的常见结果

```
/etc/systemd/system/multi-user.target.wants/foo.service
```

这个链接表达 `target` 对 `foo` 的 `Wants` 关系。不要把目录中链接的存在直接解释为当前 `active`；它只证明持久依赖配置。当前状态仍用 `is-active`。

⑤ [边界] 恢复目标和启动链深度留给第 30 章

本章只建立 `target`、默认目标和 `isolate` 的对象模型。内核参数、GRUB、`initramfs`、`rescue/emergency` 登录、根文件系统修复和启动链故障归《启动链、GRUB、Target 与系统恢复》。

[Cheatsheet] `target` 是依赖集合；`get/set-default` 管持久默认目标；`start` 不等于 `isolate`；`isolate` 会停止不属于新图的 `unit`，风险高；`wants` 目录证明持久关系，不证明当前 `active`。

[知识专题] 系统 Manager、用户 Manager 与 `loginctl`

RHEL 9 同时可以运行系统级 `systemd manager` 和每个用户自己的 `user manager`。两者管理范围、`unit` 搜索路径、权限和生命周期不同。一个服务以普通用户身份运行，不等于它属于 `user manager`。

① [知识点] 系统 `service` 的 `User=` 仍由系统 PID 1 管理

```
[Service]
User=svcwatch
```

该进程以 `svcwatch` 身份运行，但 `unit` 仍由系统 `manager` 管理：

```
systemctl status heartbeat-worker.service
```

它适合系统范围服务和开机阶段服务，不依赖该用户登录。

② [知识点] `systemctl --user` 操作独立的用户 `manager`

```
systemctl --user list-units
systemctl --user status my-worker.service
```

`user unit` 常见管理员和用户路径与系统 `unit` 不同。执行命令需要正确用户会话、运行时目录和用户 `bus`。不要用 `root` 的环境机械代替目标用户执行后就声称用户服务可用。

③ [操作] `loginctl` 查询会话、用户 `manager` 和 `linger`

```
loginctl list-users
loginctl user-status USER
loginctl show-user USER -p State,Linger,Sessions
```

`Linger=yes` 表示该用户的 `manager` 可以在没有活动登录会话时保留，并可在引导时启动。它常用于需要跨登出运行的用户服务。

④ [边界] `linger` 不是普通系统 `service` 的必需条件

系统 `manager` 中配置 `User=` 的 `service` 不依赖 `linger`。只有 `unit` 真正属于 `systemctl --user` 管理范围且要求退出登录后继续运行时，才调查 `linger`。

`rootless Podman` 的用户 `unit`、`Quadlet` 和容器持久化归第 31、32 章；本章只建立 `user manager` 接口。

⑤ [验证点] 用户服务需要跨身份和跨会话验证

目标用户下 `systemctl --user is-active/is-enabled`
→ `logindctl show-user` 检查 `Linger` 与 `State`
→ 退出登录或重启后的持久性测试 (需 live VM)
→ 应用功能验证

跨登出或重启后的持久性必须在目标 RHEL 9 主机上实际验证；当前状态与 `Linger` 属性不能代替该验收。

[Cheatsheet] 系统 `service` 中 `User=` 仍归 `PID 1`；`systemctl --user` 操作用户 `manager`；`logindctl show-user` 查 `State/Linger`；只有用户 `manager` 要跨登出运行时才需要 `linger`；容器具体流程留给 `Podman` 章节。

[诊断专题] 从症状到下一条证据：Unit 故障诊断链

`systemd` 排错的目标不是尽快找到一个能让状态变绿的命令，而是从当前证据区分：定义未加载、依赖错误、执行身份错误、程序退出、激活入口仍在、启动速率限制或功能层失败。每次只选择最能区分当前假设的下一条证据。

① [诊断] `Unit ... could not be found`

症状: `start/status` 报 `unit` 不存在
→ 当前证据: `systemctl show UNIT -p LoadState,FragmentPath`
→ 假设: 名称写错、文件不在搜索路径、主文件后缀错误、尚未 `daemon-reload`
→ 下一证据: `systemd-analyze unit-paths`; `find` 正确路径; `systemctl cat`
→ 最小修复: 修正名称或放入管理员路径, `daemon-reload`
→ 再验证: `show LoadState/FragmentPath`, 然后 `start`

不要通过复制另一个不相关 `unit` 或随意创建空文件掩盖名称错误。

② [诊断] `Loaded: masked`

症状: `unit` 存在但拒绝启动
→ 当前证据: `is-enabled` 返回 `masked`; `FragmentPath` 可能指向 `/dev/null`
→ 假设: 管理员或软件包有意屏蔽, 或遗留链接
→ 下一证据: `ls -l` 相关 `unit` 路径; 变更记录和依赖影响
→ 最小修复: 只有确认应恢复时 `unmask`
→ 再验证: `is-enabled`、`start`、持久配置与功能

`unmask` 不会自动 `enable/start`。

③ [诊断] `enabled 但当前 inactive`

可能原因:

- 当前系统尚未进入拉起它的 `target`;

- enable 后未加 `--now`，当前未启动；
- 服务启动失败后回到 inactive/failed；
- unit 受 Condition/Assert 约束；
- 依赖没有满足；
- 它是按需激活而非长期运行对象。

```
systemctl show UNIT \
    -p ActiveState,SubState,UnitFileState,Result,Conditions,ConditionResult
systemctl list-dependencies --reverse UNIT
```

不要反复 enable；先确认当前层发生了什么。

④ [诊断] active 后很快 failed 或反复重启

当前证据: ActiveState/SubState/Result/ExecMainStatus/Restart/NRestarts
→ 假设: 程序立即退出、配置错误、权限错误、Restart 导致循环
→ 下一证据: 实际 ExecStart、目标用户、工作目录、应用语法检查、unit 日志
→ 最小修复: 修正根因; 必要时 stop 并 reset-failed
→ 再验证: 稳定运行时间、NRestarts 不继续增长、功能成立

完整日志过滤归第 13 章，但本章必须知道 status 的日志片段不是全部证据。

⑤ [诊断] stop 后再次 active

当前证据: TriggeredBy、Restart、反向依赖、相关 socket/path/timer 状态
→ 假设一: 事件 unit 再激活
→ 假设二: 进程退出触发 Restart
→ 假设三: 其他 target/service 再次启动它
→ 最小修复: 关闭真正不需要的入口或调整策略
→ 再验证: 重复触发条件, 确认目标行为

不要直接 mask service，除非目标确实要求阻止所有普通激活。

⑥ [诊断] 修改 unit 后仍按旧命令或旧用户运行

当前证据: systemctl cat 与进程实际命令/UID
→ 假设: 修改了被覆盖的文件、drop-in 未加载、服务没有 restart
→ 下一证据: FragmentPath/DropInPaths; show ExecStart/User/MainPID
→ 最小修复: 改正确来源、daemon-reload、按影响 restart
→ 再验证: 新 MainPID 的 user/cmd 与功能

⑦ [诊断] 依赖已写，但服务仍早于前置对象

当前证据: `show Wants/Requires/After/Before`

- 假设: 只有需求没有顺序, 或只有顺序没有拉入事务
- 下一证据: `list-dependencies` 与排序视图
- 最小修复: 补充最小必要关系
- 再验证: 重新加载后执行受控启动, 观察两个 `unit` 的状态和业务结果

⑧ [诊断] `active` 且 `enabled`, 但业务不可用

此时 `systemd` 层已经提供两个正向证据, 下一步应转到功能链, 而不是继续 `enable/restart`:

监听端口或 `socket` 是否存在

- 本机请求是否成功
- 目标文件/数据是否持续更新
- 应用权限和安全策略是否允许
- 上游依赖是否可用

网络、`firewalld`、`SELinux` 和日志的深度分别属于其自然章节。

[Cheatsheet] 先 `status/show` 建基线; `not-found` 查名称、路径和 `reload`; `masked` 先查原因; `enabled` $\neq$ `active`; 反复重启查 `Result/Restart`; `stop` 后回来查触发和反向依赖; `active+enabled` 后转入功能证据。

[经典任务] 创建受限用户运行的 Heartbeat Service

环境

系统已经提供可执行程序:

```
/usr/local/libexec/heartbeat-worker
```

程序以前台方式长期运行, 每隔数秒更新:

```
/var/lib/heartbeat-worker/last-update
```

当前系统中:

- 不存在 `heartbeat-worker.service`;
- 不存在账号 `svcwatch`;
- 不存在数据目录 `/var/lib/heartbeat-worker`;
- 程序不需要网络;
- 不允许修改 `/usr/lib/systemd/system`。

目标终态

1. 创建不可交互登录的系统账号 `svcwatch` ;
2. 程序必须由 `svcwatch` 运行, 不得以 `root` 运行;
3. unit 名为 `heartbeat-worker.service` ;
4. 使用 `Type=simple` ;
5. `ExecStart=/usr/local/libexec/heartbeat-worker` ;
6. 异常退出时使用 `Restart=on-failure` , 等待 5 秒再重启;
7. 当前立即运行;
8. 引导进入 `multi-user.target` 时自动启动;
9. 心跳文件必须能被服务账号持续更新;
10. 不把“active”当作唯一验收证据。

限制条件

- 不使用 `chmod 777` ;
- 不关闭 SELinux;
- 不在 unit 中使用交互 Shell 的重定向技巧;
- 不通过 `root crontab` 或后台 `nohup` 代替 service;
- 不编造命令输出。

验收矩阵

层	验收要求
定义	主文件位于管理员路径, verify 无阻断性问题, FragmentPath 正确
当前	ActiveState/SubState 符合长期运行服务
持久	UnitFileState 为 enabled
身份	MainPID 的实际用户为 svcwatch
策略	Restart=on-failure, RestartSec=5s
功能	两次间隔读取心跳文件, 更新时间发生推进

[经典任务] 诊断 Service 停止后被重新激活

环境

系统存在:

```
report-gateway.socket
report-gateway.service
```

已知 service 的进程长期运行，unit 中配置 `Restart=on-failure`。管理员观察到：

1. 执行 `systemctl stop report-gateway.service` 后，service 变为 inactive；
2. 访问对应 socket 后，service 再次 active；
3. 直接向 MainPID 发送致命信号后，即使没有新连接，service 也会再次出现。

目标终态

- 分别证明第一次再激活来自 socket，第二次来自 Restart 策略；
- 停止并禁用 socket 激活；
- 保留管理员手工启动 service 的能力；
- 保留 `Restart=on-failure`；
- 不 mask service；
- 验证新连接不再自动拉起，但手工 start 后 service 可正常工作。

验收矩阵

目标	证据
激活来源	TriggeredBy、Restart、NRestarts、反向依赖
socket 入口关闭	socket inactive 且 disabled
service 可手工启动	未 masked，手工 start 成功
重启策略保留	Restart=on-failure
功能	手工启动后按题目给定方式访问成功

[参考解答] 经典任务一：Heartbeat Service

以下命令为静态推荐流程。账号参数和程序行为应以题目真实附件为准。

① 调查程序和现状

```
ls -l /usr/local/libexec/heartbeat-worker
file /usr/local/libexec/heartbeat-worker
getent passwd svcwatch
systemctl status heartbeat-worker.service --no-pager -l
```


确认程序存在且可执行，unit 和账号确实不存在。若程序是脚本，还要确认首行解释器存在。

② 创建受限系统账号和数据目录

```
useradd --system --home-dir /var/lib/heartbeat-worker \
    --shell /sbin/nologin svcwatch

install -d -o svcwatch -g svcwatch -m 0750 /var/lib/heartbeat-worker
```

`install -d` 一次明确目录、所有者和权限，避免先创建再遗漏 `chown`。不要使用 `777`。

③ 创建 unit

```
cat > /etc/systemd/system/heartbeat-worker.service <<'EOF'
[Unit]
Description=Heartbeat file worker
After=local-fs.target

[Service]
Type=simple
User=svcwatch
Group=svcwatch
WorkingDirectory=/var/lib/heartbeat-worker
ExecStart=/usr/local/libexec/heartbeat-worker
Restart=on-failure
RestartSec=5s

[Install]
WantedBy=multi-user.target
EOF
```

这里没有无意义的网络依赖；服务只依赖本地文件系统和已经准备好的目录。

④ 静态检查并让 manager 读取定义

```
systemd-analyze verify /etc/systemd/system/heartbeat-worker.service
systemctl daemon-reload
systemctl cat heartbeat-worker.service
systemctl show heartbeat-worker.service \
    -p FragmentPath,DropInPaths,Type,User,ExecStart,Restart,RestartUSec
```

若 `verify` 报告阻断性问题，应先修复，不要通过不断 `restart` 猜测。

⑤ 当前启动并配置持久激活

```
systemctl enable --now heartbeat-worker.service
```

⑥ 分层验证

```
systemctl is-active heartbeat-worker.service
systemctl is-enabled heartbeat-worker.service

systemctl show heartbeat-worker.service \
    -p LoadState,ActiveState,SubState,UnitFileState,MainPID,Result,User,Restart,RestartUsec

pid=$(systemctl show heartbeat-worker.service -p MainPID --value)
ps -p "$pid" -o pid,user,group,etimes,cmd
```

功能验证应读取两次真实时间或文件内容：

```
stat /var/lib/heartbeat-worker/last-update
sleep 6
stat /var/lib/heartbeat-worker/last-update
```

验收重点不是输出长什么样，而是第二次修改时间晚于第一次，且文件所有者、服务进程身份和 unit 状态符合要求。

⑦ 典型错误与修复方向

错误	证据	最小修复
ExecStart 路径错	status/Result、文件检查	改绝对路径，daemon-reload
账号不存在	getent、status 身份错误	创建正确系统账号
目录不可写	namei、ls、应用错误	修正最小所有权和权限
unit 已改但仍旧行为	cat/show 与进程不一致	daemon-reload 后 restart
active 但文件不更新	MainPID、用户、工作目录、应用行为	转入功能和安全证据
重启风暴	Restart/NRestarts/Result	stop，修正根因，reset-failed

[参考解答] 经典任务二：区分 Socket 激活与 Restart

① 建立初始证据

```
systemctl status report-gateway.service report-gateway.socket --no-pager -l

systemctl show report-gateway.service \
    -p ActiveState,SubState,UnitFileState,MainPID,TriggeredBy,Restart,RestartUsec,NRestarts

systemctl list-dependencies --reverse report-gateway.service
systemctl cat report-gateway.service report-gateway.socket
```

若 `TriggeredBy` 指向 `socket`，并且 `socket` 处于 `listening`，则连接触发假设成立；`Restart=on-failure` 则为异常退出后的另一条机制。

② 证明 `socket` 激活

```
systemctl stop report-gateway.service
systemctl is-active report-gateway.service
```

按题目给定方式发起一次连接，然后再次检查 `service`。不能编造测试地址或端口，必须使用任务附件提供的 `socket` 参数。

③ 证明 `Restart` 策略

先记录重启计数与 `MainPID`：

```
systemctl show report-gateway.service -p MainPID,NRestarts,Restart
```

在隔离测试环境中按题目要求模拟异常退出，再次查询 `MainPID` 与 `NRestarts`。生产环境不应为了证明机制随意杀业务进程；本章任务是受控实验设计。

④ 关闭指定激活入口

```
systemctl disable --now report-gateway.socket
systemctl is-active report-gateway.socket
systemctl is-enabled report-gateway.socket
```

不 `mask service`，因此管理员仍可手工启动：

```
systemctl is-enabled report-gateway.service
systemctl start report-gateway.service
systemctl is-active report-gateway.service
```

⑤ 再验证目标行为

1. `stop service`；
2. 发起连接，确认 `socket` 不再自动拉起；
3. 手工 `start service`；
4. 确认功能访问成功；
5. 查询 `Restart=on-failure` 未被修改。

```
systemctl show report-gateway.service -p Restart,TriggeredBy
```

⑥ 为什么不使用 mask

mask 会阻止管理员手工 start，也会改变题目明确要求保留的能力。最小修复是只关闭 socket 入口。只有目标是“任何普通方式都不得启动 service”时，才评估 mask。

[本章收束] 把 systemd 操作还原为对象和证据

本章的核心不是记住更多子命令，而是把每个动作定位到正确层：

- unit 名称与类型
 - 加载来源和合并结果
 - Wants/Requires 与 After/Before
 - ActiveState/SubState/Result
 - enabled/disabled/static/masked
 - service 的 MainPID、User、ExecStart 与 Restart
 - 触发入口和 default target
 - 对外功能验收

遇到故障时，先问“当前缺少哪一层证据”，再选择 status、show、cat、依赖查询、systemd-analyze 或 loginctl。只有 unit 定义发生变化时才需要 manager 重新加载；只有目标明确要求当前与持久同时改变时才组合 --now；只有需要阻断所有普通激活时才 mask。

为后续章节保留的接口：日志的完整检索进入《系统日志、Journal、rsyslog 与日志轮转》；timer 的调度表达进入《一次性任务、周期任务与 systemd Timer》；恢复目标与引导链进入《启动链、GRUB、Target 与系统恢复》；用户容器服务与 Quadlet 进入容器篇。

主要判断表

看到的证据	可以得出的结论	仍不能证明	下一条证据
LoadState=loaded	manager 已获得可用定义	当前已运行、定义一定符合业务目标	查 ActiveState、FragmentPath 与 drop-in
ActiveState=active	unit 达到自身活动语义	开机自动启动、端口或业务可用	查 UnitFileState 与功能结果
UnitFileState=enabled	已建立持久激活关系	当前已经运行、依赖目标当前成立	查 is-active 与目标依赖
SubState=exited	类型专用状态为已退出	一定异常或一定存在守护进程	查 Type、RemainAfterExit、Result
Restart=on-failure	异常退出可能触发重启	socket/path/timer 不会再次激活	查 TriggeredBy 与反向依赖
systemd-analyze verify 无阻断错误	静态语法和部分引用可接受	运行时权限、环境与业务功能正确	实际启动并做身份、功能验收

工作方法与向下一章交接

处理 `systemd` 问题时，先固定 `unit` 的完整名称，再按“来源 - 状态 - 关系 - 进程 - 功能”收集证据。修改时优先做最小覆盖，修改后只在定义层发生变化时执行 `daemon-reload`，再根据目标选择 `start`、`restart` 或应用自身的 `reload`。不要用一次 `restart` 掩盖对象、依赖或权限问题。

本章到此只使用 `systemctl status` 中的少量近期信息作为诊断入口。下一章《系统日志、Journal、rsyslog 与日志轮转》将接手完整日志检索、启动轮次、时间范围、优先级与持久化问题；`timer` 的日历表达式与执行验收则留给第 15 章。

终章 Cheatsheet

```
# 对象、状态、来源
systemctl status UNIT --no-pager -l
systemctl show UNIT -p LoadState,ActiveState,SubState,UnitFileState,MainPID,Result
systemctl cat UNIT

# 当前与持久
systemctl start|stop|restart|reload UNIT
systemctl enable|disable UNIT
systemctl enable --now UNIT
systemctl mask|unmask UNIT

# 关系与激活
systemctl list-dependencies UNIT
systemctl list-dependencies --reverse UNIT
systemctl show UNIT -p Wants,Requires,After,Before,TriggeredBy,Restart

# 定义与静态检查
systemd-analyze unit-paths
systemd-analyze verify /etc/systemd/system/UNIT
systemctl daemon-reload

# 目标与用户 manager
systemctl get-default
systemctl set-default multi-user.target
loginctl show-user USER -p State,Linger,Sessions
```

最终判断：

```
定义正确
+ 当前状态正确
+ 持久状态正确
+ 进程身份正确
+ 功能结果正确
= 本章任务终态成立
```